

[CS230] Sequence Models: Word2Vec & GloVe

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-9. Deep Learning Fundamentals & CNNs - *Completed*

Chapter 10. Sequence Models (Current Part)

- 10.1-10.5 RNNs, LSTM, Word Representation - *Completed*
- **10.6 Word Embedding Algorithms**
 - * The Philosophy: Distributional Hypothesis
 - * Word2Vec: Skip-gram & Negative Sampling
 - * Word2Vec: CBOW (Continuous Bag of Words)
 - * GloVe (Global Vectors)
- 10.7 Sentiment Classification - *Upcoming*

Chapter 11. Attention Mechanism - *Next Part*

지난 시간 복습 및 연결

지난 시간에 우리는 단어를 벡터로 바꾸는 임베딩의 개념을 배웠습니다. '왕'과 '남자'가 가깝다는 것을 알았죠. 그렇다면 이 마법 같은 벡터 값들은 어떻게 구할까요? 언어학자 존 퍼스는 말했습니다. "단어의 의미는 그 단어의 친구들(주변 단어)을 보면 알 수 있다." 오늘 배울 알고리즘들은 이 철학을 구현한 것입니다. 방대한 텍스트를 읽으며 단어의 관계를 파악하고 의미를 학습하는 Word2Vec과 GloVe에 대해 알아보니다.

1 Unit Overview

□ 핵심 목표

이 단원은 단어 임베딩을 학습하는 대표적인 두 가지 알고리즘을 다룹니다.

- **Skip-gram:** 중심 단어로 주변 단어를 예측하며 학습하는 방식을 이해합니다.
- **Negative Sampling:** 거대한 Softmax 연산을 피하고 속도를 높이는 최적화 기법을 배웁니다.
- **CBOW:** 주변 단어로 중심 단어를 예측하는 방식과 Skip-gram의 차이를 비교합니다.
- **GloVe:** 전체 말뭉치의 동시 등장 행렬을 활용하는 통계적 방법을 파악합니다.

2 Essential Terminology

알고리즘	방식	특징
Skip-gram	중심 → 주변 예측	희귀 단어 학습에 유리함 (널리 쓰임).
CBOW	주변 → 중심 예측	학습 속도가 빠름.
Negative Sampling	이진 분류 문제로 변환	10만 개 클래스 분류를 O/X 문제로 바꿈.
GloVe	행렬 분해 (Matrix Factorization)	전체 통계 정보를 직접 활용함.

3 Core Concepts: 친구를 보면 너를 안다

3.1 1. Word2Vec: Skip-gram Model

우리는 라벨이 없는 텍스트를 지도 학습(Supervised Learning) 문제로 바꿉니다. 문장: "I want a glass of orange juice to drink."

- **Input (Context):** orange (중심 단어)
- **Target:** juice (주변 단어)
- **Task:** "orange가 나왔을 때, 주변에 juice가 나올 확률은?"

3.2 2. The Problem with Softmax

$$P(t|c) = \frac{e^{\theta_t^T e_c}}{\sum_{j=1}^V e^{\theta_j^T e_c}}$$

분모의 합($\sum$)을 구하려면 사전(Vocabulary)에 있는 10만 개 단어를 다 계산해야 합니다. 학습 한번 할 때마다 10만 번 연산? 너무 느립니다.

3.3 3. The Solution: Negative Sampling

구글 연구팀은 문제를 바꿨습니다. "10만 개 중 하나 맞추기" → "이게 진짜 쌍(Pair)이냐 가짜냐(Binary)?"

- **Positive Pair (진짜):** (orange, juice) → Label 1
- **Negative Pair (가짜):** (orange, king), (orange, book) ... → Label 0

이제 1개의 진짜와 k 개의 가짜(보통 5 20개)만 계산하면 됩니다. 연산량이 1/10000로 줄어듭니다. 이것이 Word2Vec의 핵심입니다.

4 GloVe (Global Vectors)

Word2Vec은 윈도우를 슬라이딩하며 국소적(Local) 정보만 봅니다. GloVe는 전체 통계(Global Statistics)를 봅니다.

□ Co-occurrence Matrix (동시 등장 행렬) (수학적 원리)

X_{ij} = 단어 i 주변에 단어 j 가 등장한 횟수

GloVe의 목표는 임베딩 벡터의 내적이 이 횟수의 로그값과 비슷해지는 것입니다.

$$w_i^T w_j + b_i + b_j \approx \log(X_{ij})$$

5 Implementation: Gensim Word2Vec

실무에서는 직접 구현하기보다 최적화된 라이브러리 ‘Gensim’을 씁니다.

```
1 from gensim.models import Word2Vec
2
3 # 1. 학습 데이터 토큰화된(문장 리스트)
4 sentences = [
5     ['I', 'love', 'machine', 'learning'],
6     ['deep', 'learning', 'is', 'fun'],
7     ['machine', 'learning', 'is', 'hard'],
8     ['orange', 'juice', 'is', 'delicious']
9 ]
10
11 # 2. 모델 학습
12 # vector_size: 임베딩 차원 (100~300)
13 # window: 주변 단어 범위 (5)
14 # sg: 1=Skip-gram, 0=CBOW
15 model = Word2Vec(sentences, vector_size=10, window=2, min_count=1, sg=1)
16
17 # 3. 결과 확인
18 vector = model.wv['machine']
19 print(f"Vector:\n{vector}")
20
21 # 4. 유사도 확인 핵심()
22 similar = model.wv.most_similar('deep')
23 print(f"Similar to 'deep': {similar}")
```

Listing 1: Word2Vec Training with Gensim

6 FAQ & Pitfalls

데이터 양이 중요합니다 (오해 방지 가이드)

위의 예제처럼 문장 4개로는 아무것도 못 배웁니다. Word2Vec이 제대로 작동하려면 위키피디아 전체 같은 **대용량 텍스트**가 필요합니다. 실무에서는 보통 구글이나 페이스북이 미리 학습해둔 모델(Pre-trained)을 다운받아 씁니다.

Q. Word2Vec과 GloVe 중 뭐가 더 좋나요?

A. 비슷합니다. 어떤 데이터셋에서는 Word2Vec이, 어떤 곳에서는 GloVe가 낫습니다. 보통은 둘 다 써보고 성능 좋은 걸 택하거나, 최근에는 BERT 같은 문맥 기반 임베딩을 씁니다.

다음 단계 (Next Step)

이제 우리는 단어 하나하나를 의미 있는 벡터로 바꾸는 법을 알았습니다. 하지만 "I ate an apple"과 "An apple was eaten by me"는 단어 순서가 다르지만 의미는 같습니다. 반면 RNN은 길이가 길어지면 앞을 잊어버리는 문제가 여전합니다.

다음 시간에는 입력 시퀀스 전체를 보고 번역하거나 요약하는 [Seq2Seq] 모델과, 긴 문장에서 중요한 단어에 집중하게 만드는 [Attention Mechanism]을 배우겠습니다. 이것이 현대 AI의 정점인 Transformer로 가는 마지막 관문입니다.

□ 단원 요약 (Cheat Sheet)

1. **Skip-gram:** 중심 단어로 주변 단어를 예측한다. (희귀 단어에 강함)
2. **Negative Sampling:** 전체 Softmax 대신 O/X 문제로 바꿔 속도를 높인다.
3. **GloVe:** 전체 말뭉치의 통계(동시 등장 횟수)를 직접 활용한다.
4. **Tip:** 데이터가 적을 땐 Pre-trained 모델을 써라.